Brain Tumor MRI Classification — TensorFlow vs PyTorch, served with FastAPI

Course: AI Platforms, Fall 2026 — Faculty of Computers and Data Science, Alexandria University
Assignment: 1 | **Team ID:** T20
Team members: Seif (Role 1 — data, FastAPI, report), Ahmed Refaat (Role 2 — TensorFlow), Moatasem (Role 3 — PyTorch)

---

1. Introduction

---

2. Dataset and split

Source: Brain Tumor MRI Dataset (Kaggle, Masoud Nickparvar) — `Training/` and `Testing/` folders, 4 class folders each. All files are JPEG.

Split	glioma	meningioma	notumor	pituitary	Total
Train	1,190	1,190	1,190	1,190	**4,760**
Validation	210	210	210	210	**840**
Test (official `Testing/`)	400	400	400	400	**1,600**
Total	1,600	1,600	1,600	1,600	**7,200**

- The dataset is **perfectly class-balanced** (1,400 trainval + 400 test per class); no class weights were needed.
- **Split rule (contract):** the official `Testing/` folder is the test set; a stratified 15% of `Training/` (seed 42) is carved as validation. One shared `split_manifest.csv` is used verbatim by both frameworks; no re-splitting was performed.
- **Image properties (EDA):** 0 corrupt files; **mixed color modes** — 3,067 grayscale (mode L), 4,129 RGB, 3 RGBA, 1 palette → all pipelines call `convert("RGB")`. **447 distinct resolutions** (5,014 of 7,200 are 512×512); all images are resized to 224×224.
- **File-name quirk:** meningioma test images are named `Te-aug-*`, indicating the source dataset stores augmented copies for that class; we treat them as given test data.

Duplicate and leakage analysis (SHA-256 over all 7,200 files)

Check	Result
Exact duplicate groups (same hash)	153 groups, 340 files (all within-class; 0 cross-label conflicts)
Test set leaking into train/val	**0 groups — test set is clean** ✅
Train ↔ val duplicates	42 groups; **44 of 840 validation images (5.2%)** are exact copies of training images (30 notumor, 10 pituitary, 4 meningioma)

Decision: per the shared contract the split was not altered. The 5.2% train/val duplication mildly optimizes *validation* metrics (model selection), but the headline **test numbers are unaffected**, and both frameworks use the identical manifest, so the framework comparison remains fair. See Limitations (§9).

---

3. Preprocessing and augmentation

---

4. PyTorch implementation

Deliverable: fill from `notebooks/pytorch_T20.ipynb` + `results/metrics_pytorch.json`.

---

5. TensorFlow implementation

Deliverable: fill from `notebooks/tensorflow_T20.ipynb` + `results/metrics_tensorflow.json`.

---

6. Interpretability (GradCAM)

Deliverable: GradCAM figures from both notebooks (identical image list).

---

7. Framework comparison

---

8. Architecture / hyperparameter / strategy comparison

---

9. Limitations

---

10. Deployment (which model was served and why)

---

11. Example API requests

```
$ curl -s http://localhost:8000/health
←!— PASTE REAL RESPONSE —→

$ curl -s -X POST http://localhost:8000/predict -F "file=@data/Testing/..."
←!— PASTE REAL RESPONSE —→

$ curl -s -X POST http://localhost:8000/predict -F "file=@notes.txt"
←!— PASTE REAL RESPONSE (4xx) —→
```

---

12. Appendix: environment and package versions